```
    PhoneApp.Conversations.create_sms_message(message)
  end

  defp incoming_sms_response do
    """
    <?xml version="1.0" encoding="UTF-8"?>
    <Response></Response>
    """
  end
end
```

This controller route receives a payload and then persists the received SMS message to the database. This is just like the other controllers that we created in the previous chapter, except it replies with an XML payload instead of JSON or HTML. (XML is the format that Twilio requires for webhook responses.)

In addition, you need to add this route to the PhoneAppWeb.Router module. Add the following code to lib/phone_app_web/router.ex:

phone_app/lib/phone_app_web/router.ex

```
scope "/webhook", PhoneAppWeb.Webhook do
  pipe_through [:api]

  post "/sms", TwilioController, :sms
end
```

This routes POST requests to /webhook/sms to the PhoneAppWeb.Webhook.TwilioController.sms/3 function.

Use the App

Start the PhoneApp project using mix phx.server. In addition, start the MockServer project—from earlier in this chapter—with the same command.

Open two different tabs in your browser. Load the first to http://localhost:4004/messages/new and the second to http://localhost:4005.

On the first tab (PhoneApp), you'll see the message form to deliver a new message. Enter your phone number and type a message. In the second tab (MockServer), you'll see the message appears instantly in the table. There is a "reply" button that lets you send a message in response.

We don't cover real-time in this book, so you have to refresh the PhoneApp page to see the response come through. However, the MockServer app is built using LiveView, so messages will instantly appear without a page refresh. You can see the potential here even if we didn't implement it in our own project. We won't dedicate a chapter to LiveView in this book, but we'll touch on it in the final chapter.